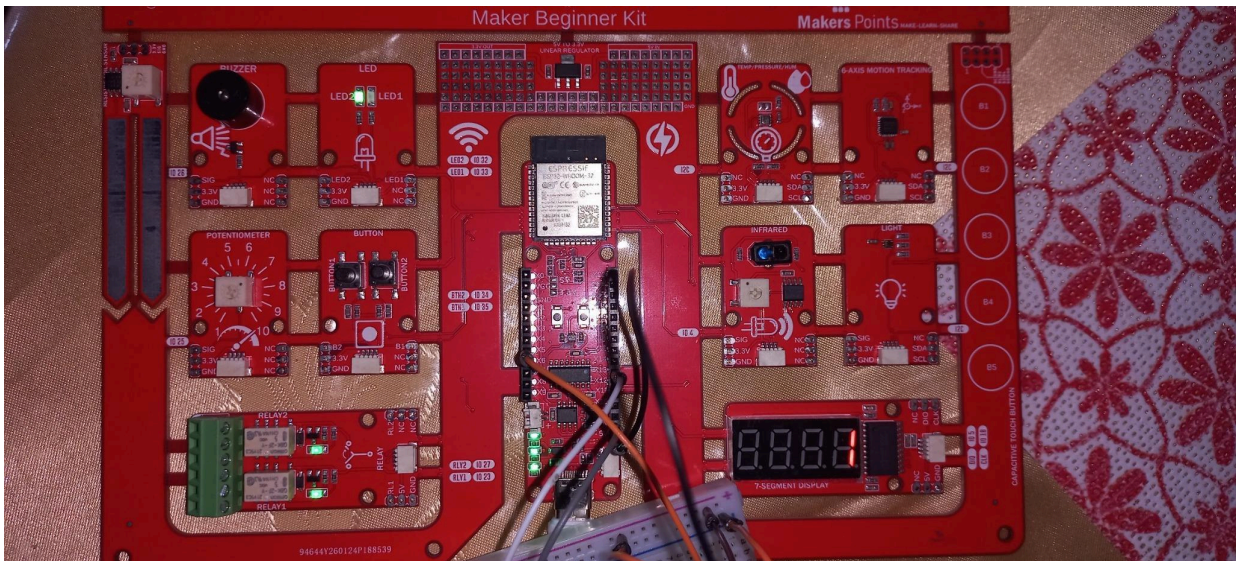


PROJET FINAL - SEMAINE 1

SMART ROBOT SAFETY CONTROLLER

Documentation du prototype fonctionnel



Prototype physique sur la carte Maker Point Beginner Kit avec ESP32 et modules câblés.

Projet	Smart Robot Safety Controller
Plateforme	Maker Point Beginner Kit + ESP32
Document	Livrable de documentation - prototype, architecture, fonctionnement et tests

1. Objet du document

Ce document présente le prototype réalisé pour le projet final de la semaine 1 : un contrôleur de sécurité pour robot fondé sur une carte ESP32, des capteurs physiques, une interface de supervision Pygame et un journal de télémétrie. Il regroupe uniquement les éléments nécessaires à la compréhension et à l'évaluation du travail réalisé.

Livrables couverts par cette documentation

Livrable demandé	Contenu présenté
Prototype fonctionnel sur la plaque	Montage physique et démonstration des états du système
Schéma bloc du système	Architecture générale et circulation des données
Code ESP32	Lecture, décision, sorties et publication MQTT
Journal de test CSV	Historique des mesures exportable depuis Google Sheets
Tableau de tests	Scénarios, résultats observés et statut
Mini-présentation 5 minutes	Déroulé court de démonstration

Objectif fonctionnel du prototype

Le système lit les capteurs, détermine localement un état de sécurité (NORMAL, WARNING, STOP ou FAULT), commande les sorties physiques, puis transmet la télémétrie via MQTT vers l'interface Pygame et le flux d'enregistrement.

- Détecter un obstacle par capteur infrarouge.
- Utiliser le potentiomètre comme simulation de vitesse.
- Détecter un choc à partir du MPU6050.
- Permettre un arrêt physique et un arrêt distant.
- Mesurer l'humidité avec le BME280 et l'inclure dans la télémétrie.
- Visualiser et enregistrer les états et mesures du prototype.

2. Prototype matériel et composants utilisés

Élément	Rôle dans le prototype
Carte ESP32 / Maker Point Beginner Kit	Exécution du programme, lecture des entrées et commande des sorties
Capteur infrarouge (IR)	Détection d'un obstacle
Potentiomètre externe	Simulation de la vitesse du robot
MPU6050	Mesure des accélérations et détection de choc
BME280	Mesure de l'humidité transmise dans la télémétrie
Bouton STOP	Déclenchement d'un arrêt physique
LEDs, buzzer, relais, afficheur	Signalisation et action associées à l'état du système

Vues du montage réalisé



Figure 1 - Vue générale de la carte et vue détaillée du câblage externe sur breadboard.

Le montage externe visible comprend notamment le potentiomètre et la LED raccordés à la carte. Les modules intégrés de la carte sont utilisés pour les entrées et sorties du scénario de sécurité.

3. Schéma bloc du système

Le schéma ci-dessous représente l'architecture du prototype et distingue le flux de données de télémétrie du flux de commande distant.

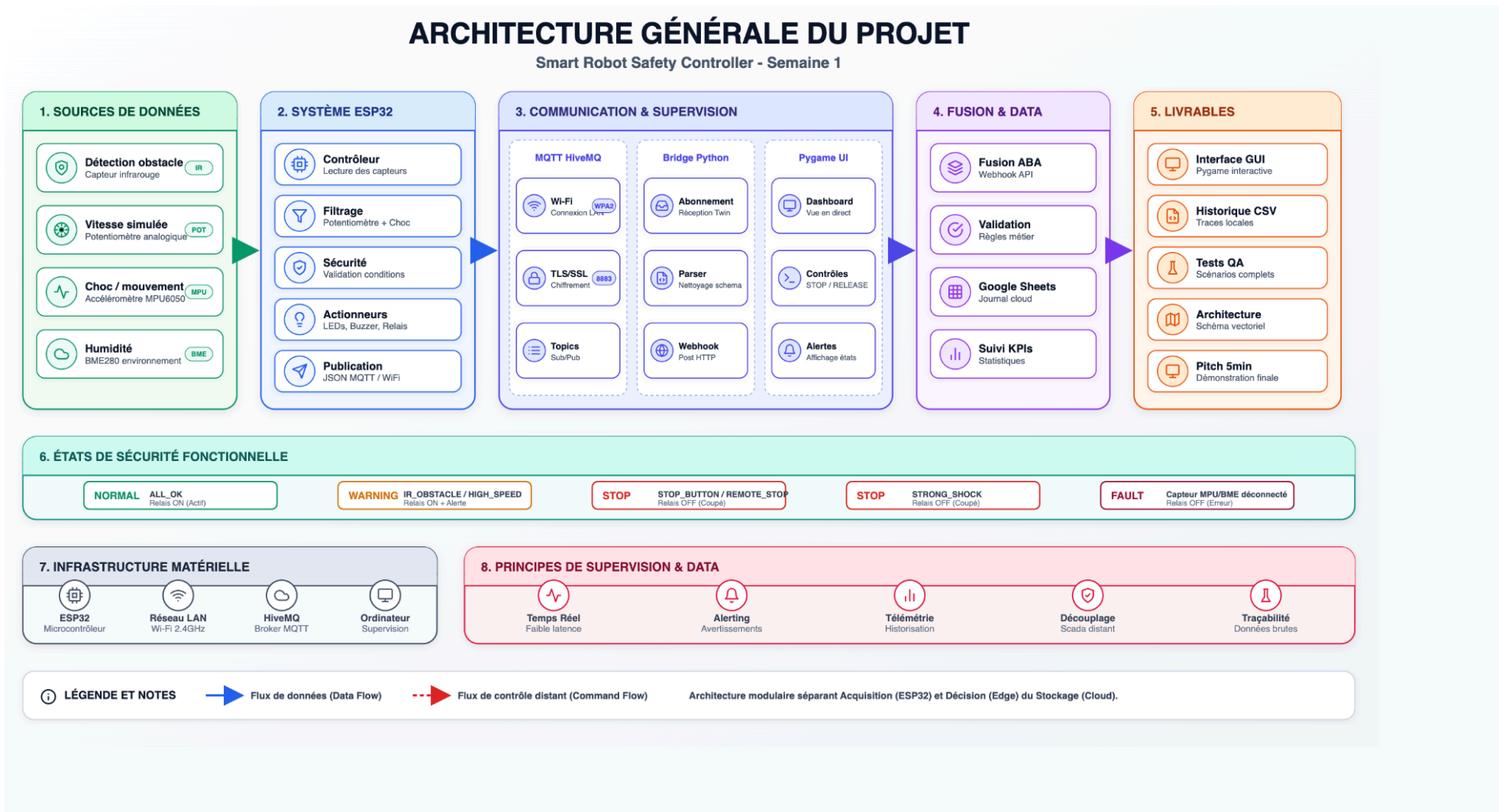


Figure 2 - Architecture générale du Smart Robot Safety Controller.

4. Fonctionnement du système

Chaîne de fonctionnement

Étape	Bloc	Description
1	Acquisition	Lecture IR, potentiomètre, bouton STOP, MPU6050 et humidité BME280.
2	Décision ESP32	Calcul local de l'état selon les conditions de sécurité.
3	Sorties physiques	LEDs, buzzer, relais et afficheur indiquent ou appliquent la décision.
4	Télémétrie	Publication du JSON sur MQTT pour la supervision.
5	Supervision et journal	Pygame affiche les mesures ; Fusion transmet les données vers Google Sheets.

Logique locale de sécurité

État	Condition	Action
NORMAL	Aucune condition de risque détectée	Relais ON ; fonctionnement autorisé
WARNING	Obstacle IR ou vitesse simulée élevée	Alerte ; relais ON
STOP	Bouton STOP, commande distante ou choc important	Relais OFF ; arrêt de sécurité
FAULT	Défaut de mesure critique du système	Relais OFF ; signalisation de défaut

Remarque : l'humidité est mesurée et transmise dans la télémétrie. Dans la version documentée, elle n'est pas utilisée pour changer l'état de sécurité du robot.

Seuils utilisés dans le scénario

Condition	Seuil / règle	Résultat attendu
Obstacle IR	IR = 1	WARNING / IR_OBSTACLE
Vitesse simulée élevée	pot_filtered >= 3000	WARNING / HIGH_SPEED
Choc important	shock_delta >= 0,80	STOP / STRONG_SHOCK

5. Communication et télémétrie

La carte ESP32 envoie périodiquement un message JSON contenant les mesures et l'état du système. Le programme Python reçoit le message MQTT, l'affiche dans l'interface et le transfère au webhook Fusion pour l'enregistrement.

Topics MQTT utilisés

Topic	Sens	Usage
hafida/robot/twin/telemetry	ESP32 vers Python	Mesures et état du système
hafida/robot/twin/command	Python vers ESP32	Commandes STOP, RELEASE_STOP et STATUS

Données enregistrées

Champ(s)	Signification
device	Identifiant du prototype
time_ms	Temps de fonctionnement ESP32
ir / obstacle	Détection d'obstacle
pot_raw / pot_filtered	Vitesse simulée
stop_pressed	État du bouton STOP
ax, ay, az / shock_delta	Mesures de choc
humidity_pct / bme_available	Humidité et présence du BME280
state / reason / relay	Décision et action de sécurité
remote_stop / rssi	Commande distante et réseau

```

* Executing task: platformio device monitor
.01,"az":1.18,"shock_delta":0.17,"humidity_pct":41.74,"bme_available":true,"state":"NORMAL","reason":"ALL_OK","relay":"ON","remote_stop":false,"rssi":
-52}
{"device":"hafida-smart-robot-safety","time_ms":65192,"ir":0,"obstacle":false,"pot_raw":103,"pot_filtered":103,"stop_pressed":false,"ax":0.05,"ay":-0
.01,"az":1.17,"shock_delta":0.17,"humidity_pct":41.71,"bme_available":true,"state":"NORMAL","reason":"ALL_OK","relay":"ON","remote_stop":false,"rssi"
:-52}
{"device":"hafida-smart-robot-safety","time_ms":66216,"ir":0,"obstacle":false,"pot_raw":103,"pot_filtered":99,"stop_pressed":false,"ax":0.04,"ay":-0
.02,"az":1.17,"shock_delta":0.17,"humidity_pct":41.71,"bme_available":true,"state":"NORMAL","reason":"ALL_OK","relay":"ON","remote_stop":false,"rssi":
-54}
{"device":"hafida-smart-robot-safety","time_ms":67240,"ir":0,"obstacle":false,"pot_raw":103,"pot_filtered":100,"stop_pressed":false,"ax":0.04,"ay":-0
.01,"az":1.17,"shock_delta":0.17,"humidity_pct":41.71,"bme_available":true,"state":"NORMAL","reason":"ALL_OK","relay":"ON","remote_stop":false,"rssi"
:-53}
{"device":"hafida-smart-robot-safety","time_ms":68263,"ir":0,"obstacle":false,"pot_raw":106,"pot_filtered":103,"stop_pressed":false,"ax":0.05,"ay":-0
.01,"az":1.17,"shock_delta":0.17,"humidity_pct":41.67,"bme_available":true,"state":"NORMAL","reason":"ALL_OK","relay":"ON","remote_stop":false,"rssi"
:-54}
{"device":"hafida-smart-robot-safety","time_ms":69286,"ir":0,"obstacle":false,"pot_raw":105,"pot_filtered":103,"stop_pressed":false,"ax":0.05,"ay":-0

```

Figure 3 - Moniteur PlatformIO : messages JSON de télémétrie incluant humidity_pct et bme_available.

6. Supervision Pygame et intégration des données

L'interface Pygame sert à visualiser l'état calculé par l'ESP32, les mesures principales et les commandes distantes. Elle joue également le rôle de passerelle vers Fusion pour le journal de télémétrie.

Fonctions visibles dans l'interface

Fonction	Description
Supervision	État NORMAL / WARNING / STOP / FAULT et raison associée
Mesures	IR, potentiomètre, choc, accélérations et RSSI affichés en temps réel
Action	Commandes STOP, RELEASE STOP et REQUEST STATUS
Intégration	Affichage de l'état d'envoi HTTP vers Fusion

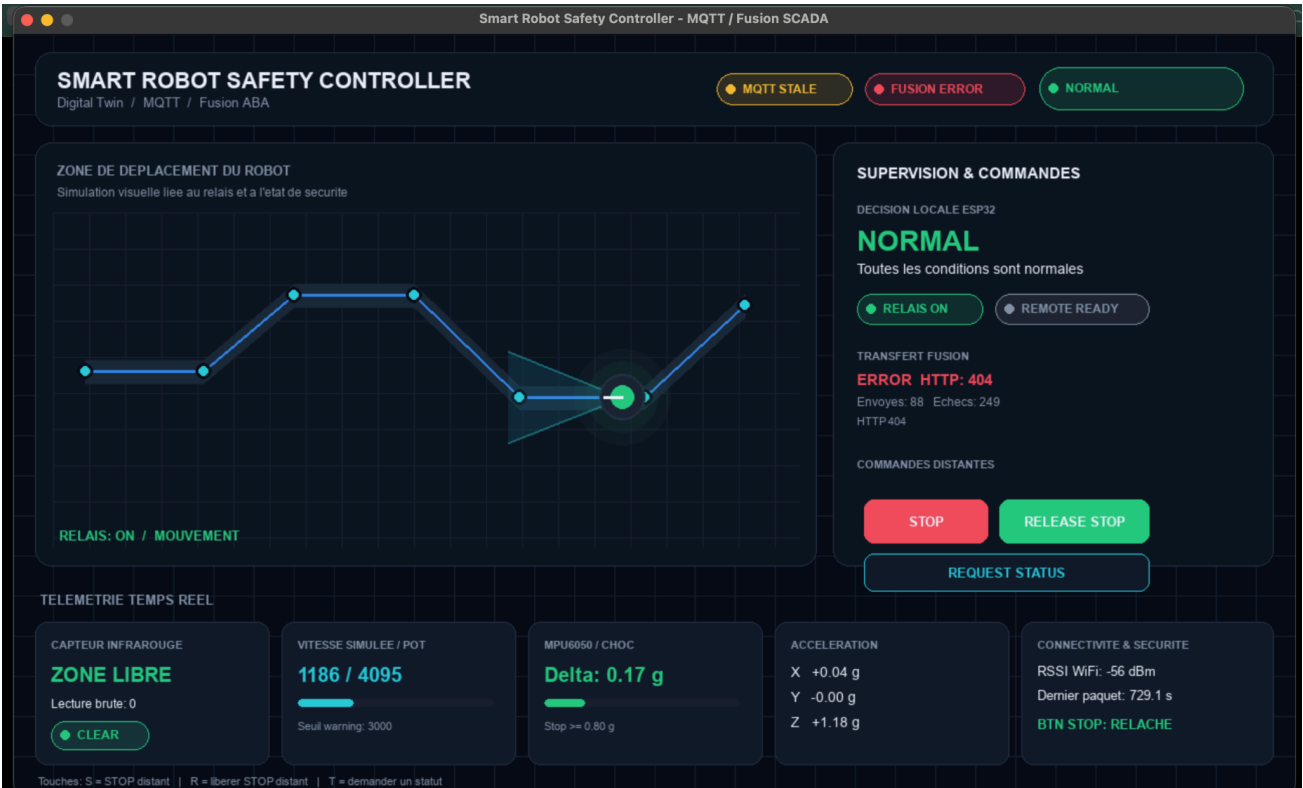


Figure 4 - Interface Pygame SCADA pendant une exécution du prototype.

Point à vérifier avant la démonstration : la capture de l'interface affiche un statut Fusion HTTP 404 au moment de la prise de vue. Vérifier l'URL du webhook et l'état du workflow avant la présentation finale.

7. Journal de télémétrie et tableau de tests

Les mesures sont stockées dans Google Sheets puis peuvent être exportées en CSV pour constituer le journal de test demandé. Les résultats ci-dessous reprennent les cas observés dans les données transmises pendant les essais.

Structure du journal CSV

```
received_at, device, time_ms, ir, obstacle, pot_raw, pot_filtered, stop_pressed,
ax, ay, az, shock_delta, humidity_pct, bme_available, state, reason, relay,
remote_stop, rssi
```

Tests documentés

ID	Test	Action	Résultat attendu	Preuve	Statut
T01	État normal	Aucun obstacle, vitesse basse	NORMAL / ALL_OK / ON	Observé	Réussi
T02	Obstacle IR	Objet devant le capteur IR	WARNING / IR_OBSTACLE / ON	Observé	Réussi
T03	Vitesse élevée	Potentiomètre au-dessus du seuil	WARNING / HIGH_SPEED / ON	Observé	Réussi
T04	Humidité	Lecture BME280	humidity_pct renseigné ; BME TRUE	42,05 à 43,62 %	Réussi
T05	STOP distant	Bouton STOP Pygame	STOP / REMOTE_STOP / OFF	Observé	Réussi
T06	STOP physique	Bouton STOP sur la carte	STOP / STOP_BUTTON / OFF	Observé	Réussi
T07	Choc fort	Déplacer le MPU6050 au-delà du seuil	STOP / STRONG_SHOCK / OFF	Non fourni	À compléter
T08	FAULT	Provoquer un défaut capteur contrôlé	FAULT / Relay OFF	Non fourni	À compléter

Exemples de données observées

Scénario	Valeurs enregistrées
Obstacle IR	ir = TRUE ; state = WARNING ; reason = IR_OBSTACLE ; relay = ON
Vitesse élevée	pot_filtered = 3097 à 4094 ; state = WARNING ; reason = HIGH_SPEED
Humidité	bme_available = TRUE ; humidity_pct = 42,05 à 43,62
Arrêt distant	state = STOP ; reason = REMOTE_STOP ; relay = OFF ; remote_stop = TRUE

8. Organisation du code et fichiers à remettre

Organisation fonctionnelle du code ESP32

Bloc du code	Rôle
Initialisation	Capteurs, sorties, I2C, Wi-Fi et MQTT
Lecture et filtrage	Acquisition IR, potentiomètre, bouton, MPU6050 et BME280
Décision	Priorité FAULT > STOP > WARNING > NORMAL
Sorties	LEDs, buzzer, relais et afficheur 7 segments
Télémétrie	Construction du JSON et publication MQTT
Commandes	Traitement de STOP, RELEASE_STOP et STATUS

Fichier vidéo (<https://drive.google.com/file/d/1ovy-ZIZADbcmk9YbR4GHcpmyqeQ1rFIZ/view?usp=sharing>).